Preserving State in

Web Applications

ASP.NET State Management & List-Bound Controls

Topics Covered

- Cookies & Session State
- Out-of-Process State Server
- SQL Server Session Storage
- Cookieless Session IDs
- DataList & Repeater Controls
- List-Bound Controls Overview

Preserving State in Web Applications

Understanding the Stateless HTTP Problem

HTTP is Stateless

Each HTTP request is independent. The server does not remember previous interactions with the same client.

The Challenge

Web apps need to track users across pages — shopping carts, login sessions, preferences & form data.

State Management

Techniques like Cookies, Session State, and Application State solve this by persisting data across requests.

Page-Level State (View State) — ASP.NET stores control values in a hidden field (__VIEWSTATE) on each page. This preserves data between postbacks without any server-side storage. Enabled by default; can be disabled per control.

Using Cookies to Preserve State

Small pieces of data stored on the client browser

Setting a Cookie (C# / ASP.NET)

```
HttpCookie cookie = new HttpCookie("user");
cookie["name"] = "Pravin";
cookie.Expires = DateTime.Now.AddDays(7);
Response.Cookies.Add(cookie);

// Reading the Cookie
string name = Request.Cookies["user"]["name"];
```

Persistent

Survive browser restarts; stored on disk until expiry date.

Session

In-memory only; deleted when browser closes (no Expires set).

Limitations

Max ~4KB per cookie; browsers limit ~50 cookies per domain.

Security

Use HttpOnly & Secure flags to protect sensitive data.

⚡ *Cookies flow automatically in every HTTP request via the Cookie header.*

ASP.NET Session State

Server-side per-user data storage keyed by a Session ID

Storing & Retrieving

```
// Store
Session["username"] = "Alice";
Session["cart"] = cartObject;

// Retrieve
string user = (string)Session["username"];

// Remove
Session.Remove("cart");
```

Key Properties & Methods

`SessionID`	Unique 120-bit identifier per user session
`Timeout`	Default 20 min of inactivity (configurable)
`Count`	Number of items in current session
`Abandon()`	Immediately ends the session

Session data lives in server memory (by default) — not sent to the client.

Storing Objects in Session State

Persisting complex types across requests

Serializable Class + Session Storage

```
[Serializable]
public class ShoppingCart {
  public List<Item> Items;
  public decimal Total;
}

// Store
ShoppingCart cart = new ShoppingCart();
cart.Items.Add(new Item("Laptop", 55000));
Session["Cart"] = cart;

// Retrieve
ShoppingCart c =
  (ShoppingCart)Session["Cart"];
```

⚠️ Must be Serializable

Required when using out-of-process or SQL Server session modes.

🔄 Deep Copy on Retrieval

Always cast the retrieved object to its original type before use.

💾 Memory Impact

Large objects in session can exhaust server memory. Use caching wisely.

🗑️ Clear on Logout

Call Session.Clear() or Session.Abandon() to free memory on logout.

Configuring Session State

web.config <sessionState> element settings

web.config — sessionState Configuration

```
<sessionState mode="InProc" cookieless="false" timeout="20"
  stateConnectionString="tcpip=127.0.0.1:42424"
  sqlConnectionString="data source=localhost;Integrated Security=SSPI" />
```

Mode	Location	Persists Restart?	Performance	Use Case
InProc	Web server memory	❌ No	⚡ Fastest	Dev / Single-server
StateServer	ASP.NET State Service	❌ No	🔸 Moderate	Web farms
SQLServer	SQL Server DB	✅ Yes	🔸 Slower	Mission-critical apps
Custom	Custom provider	Depends	Varies	Special storage needs
Off	Disabled	—	N/A	No session needed

Setting Up an Out-of-Process State Server

ASP.NET State Service for multi-server deployments

01

Start ASP.NET State Service

```
net start aspnet_state  (or via Services.msc)
Sets service to "Automatic" startup type.
```

02

Configure web.config

```
<sessionState mode="StateServer"
  stateConnectionString="tcpip=127.0.0.1:42424"
/>
```

03

Open Firewall Port

```
Default port 42424 must be open between
web servers and the state server host.
```

04

Mark Objects [Serializable]

```
All session objects MUST be serializable
because they leave the web server process.
```

⚙️ *State Server runs as a Windows Service (aspnet_state.exe) — separate from IIS.*

Storing Session State in SQL Server

Durable, scalable session storage using a database

Setup Steps

```
1. Run aspnet_regsql.exe to create
   ASPState database in SQL Server.

2. Configure web.config:
   mode="SQLServer"
   sqlConnectionString=
   "data source=.;
    Integrated Security=SSPI"

3. Mark all session objects as
   [Serializable]

4. Ensure SQL Server is running
   and accessible from web server.
```

✅ Survives Restarts

Session data persists through app or server restarts.

🔄 Web Farm Support

Multiple web servers share one session store seamlessly.

📊 Auditable

DBA can query/monitor session data directly in SQL.

⚠️ Performance Cost

Slower than InProc due to DB I/O; use wisely.

Using Cookieless Session IDs

Embedding Session ID in the URL for cookie-disabled browsers

How It Works:

Session ID is embedded directly in the URL path instead of a cookie. e.g., http://site.com/(lit3py55t21z5v55vlm25s55)/page.aspx

Configuration

```
<!-- Always use cookieless -->
<sessionState cookieless="true" />

<!-- Auto-detect -->
<sessionState cookieless="AutoDetect" />

<!-- UseUri / UseCookies / UseDeviceProfile -->
```

CON

Session ID visible in URL — security risk
Breaks if URL is shared or bookmarked
Can cause issues with relative links

⚠️ **Security Note:**

Session IDs in URLs can be stolen via Referer headers. Use HTTPS and regenerate IDs post-login. Avoid for sensitive applications.

Application State

Shared, global data accessible by all users and sessions

Feature	Session State	Application State
Scope	Per user	All users (global)
Storage	Server / DB	Web server memory
Lifetime	Until timeout / abandon	App lifetime
Thread Safety	Per-user (safe)	Requires Lock/Unlock
Use Case	Cart, user prefs	Hit counters, global config

Application State Usage

```
// Lock before writing
Application.Lock();
Application["hits"] =
  (int)Application["hits"] + 1;
Application.UnLock();

// Read (no lock needed)
int h = (int)Application["hits"];
```

1. Application state is initialized in Global.asax (Application_Start).
2. Always Lock() / UnLock() when writing to prevent race conditions.
3. Not suitable for web farm deployments — use a shared cache instead.

Overview of List-Bound Controls

Controls that bind to data sources for dynamic rendering

GridView

- Full grid with columns/rows
- Built-in sorting & paging
- Edit/Delete row support
- Auto-generated columns
- Best for tabular data

Use: Admin data tables, reports

DataList

- Table-based multi-column layout
- Customizable item templates
- Supports multi-column display
- EditItemTemplate for editing
- More layout flexibility

Use: Product grids, image galleries

Repeater

- Pure template-driven output
- Renders exactly what you define
- No default HTML/table output
- Lightweight & flexible
- Header/Footer templates

Use: Custom lists, menus, cards

All list-bound controls require a data source (DataTable, List<T>, etc.) bound via DataSource + DataBind().

Creating a Repeater Control

Template-driven, lightweight list rendering in ASP.NET

HeaderTemplate

Rendered once at the start

ItemTemplate

Rendered for each data row

AlternatingItemTemplate

Alternating rows (optional)

SeparatorTemplate

Between each item

FooterTemplate

Rendered once at the end

ASPX Markup + Code-Behind

```
<!-- ASPX Markup -->
<asp:Repeater ID="rptProducts" runat="server">
  <HeaderTemplate><ul></HeaderTemplate>
  <ItemTemplate>
    <li><%# Eval("Name") %> — <%# Eval("Price") %></li>
  </ItemTemplate>
  <FooterTemplate></ul></FooterTemplate>
</asp:Repeater>

// Code-Behind (C#)
protected void Page_Load(object sender, EventArgs e) {
  if (!IsPostBack) {
    rptProducts.DataSource = GetProducts();
    rptProducts.DataBind();
  }
}
```

Creating a DataList Control

Multi-column grid layouts with item templates

DataList — Markup & Code-Behind

```
<!-- ASPX Markup -->
<asp:DataList ID="dlProducts" runat="server"
  RepeatColumns="3"
  RepeatDirection="Horizontal"
  RepeatLayout="Table">
  <ItemTemplate>
    <div class="card">
      <b><%# Eval("Name") %></b>
      <p>₹<%# Eval("Price") %></p>
    </div>
  </ItemTemplate>
  <SelectedItemTemplate>
    <b style="color:red"><%# Eval("Name") %></b>
  </SelectedItemTemplate>
</asp:DataList>

// Code-Behind
dlProducts.DataSource = GetProducts();
dlProducts.DataBind();
```

RepeatColumns

Number of items per row (e.g., 3 for a 3-column grid).

RepeatDirection

Horizontal fills rows first; Vertical fills columns first.

RepeatLayout

Table (default, uses <table>) or Flow (uses).

EditItemTemplate

Displayed when a row is in edit mode (set EditItemIndex).

SelectedItemTemplate

Shown for the currently selected item.

Summary & Key Takeaways

01

HTTP is Stateless

Use ViewState, Cookies, Session, or Application State to maintain data.

02

Cookies

Small client-stored data; use for non-sensitive, lightweight persistence.

03

Session State

Server-side per-user store. Choose InProc, StateServer, or SQLServer mode.

04

Object Storage

Classes must be [Serializable] for out-of-process / SQL Server modes.

05

Cookieless Sessions

Embeds Session ID in URL — useful but less secure; prefer HTTPS.

06

DataList & Repeater

Template-driven controls for flexible, data-bound HTML rendering.